Title- “GovAssist: AI Chatbot for E-Governance”

Atharva Nitin Kulkarni
2023-B-02062004
B. Tech
Computer Science Engineering (CSE)

Rushikesh Kishor Pattiwar
2023-B-05102002
B. Tech
Computer Science Engineering (CSE)

Priti Shahaji Satre
2022-B-19072003B
B. Tech
Computer Science Engineering (CSE)

AJEENKYA
D Y PATIL UNIVERSITY
THE INNOVATION UNIVERSITY
School of Engineering

“GovAssist: AI Chatbot for E-Governance”

This is submitted in partial fulfilment of the requirements of the degree of Bachelor of Technology

in

Computer Science and Engineering

By

Atharva Nitin Kulkarni: 2023-B-02062004

Rushikesh Kishor Pattiwar: 2023-B-05102002

Priti Shahaji Satre: 2022-B-19072003B

Under the Supervision of

Dr. Prashant Kumbharkar

AJEENKYA
D Y PATIL UNIVERSITY
THE INNOVATION UNIVERSITY

Academic Year 2025 - 2026

School of Computer Science and Technology

Ajeenkya DY Patil University, Pune

AJEENKYA D Y PATIL UNIVERSITY
THE INNOVATION UNIVERSITY
School of Engineering

Date:

CERTIFICATE

This is to certify that the dissertation entitled **"GovAssist: AI Chatbot for E-Governance"** is a Bonafide work of "**Atharva Nitin Kulkarni (2023-B-02062004), Rushikesh Kishor Pattiwar (2023-B-05102002), Priti Shahaji Satre (2022-B-19072003B)",** submitted to the School of Computer Science and Technology, Ajeenkya D Y Patil University, Pune in partial fulfilment of the requirement for the award of the degree of **"Bachelor of Technology in Computer Science Engineering"**.

Dr. Prashant Kumbharkar

Project Supervisor

Internal-Examiner/s

External Examiner

Dr. Prashant Kumbharkar

Dean-School of Computer Science and Technology

AJEENKYA D Y PATIL UNIVERSITY
THE INNOVATION UNIVERSITY
School of Engineering

Date:

Supervisor's Certificate

This is to certify that the dissertation entitled **"GovAssist: AI Chatbot for E-Governance"** submitted by "**Atharva Nitin Kulkarni (2023-B-02062004), Rushikesh Kishor Pattiwar (2023-B-05102002), Priti Shahaji Satre (2022-B-19072003B)",** is a record of original work carried out by him/her under my supervision and guidance in partial fulfillment of the requirements of the degree of **Bachelor of Technology in Computer Science Engineering -CSE** at **School of Computer Science and Technology**, **Ajeenkya DY Patil University, Pune, Maharashtra- 412105**. Neither this dissertation nor any part of it has been submitted earlier for any degree or diploma to any institute or university in India or abroad.

Dr. Prashant Kumbharkar
Supervisor

AJEENKYA D Y PATIL UNIVERSITY
THE INNOVATION UNIVERSITY
School of Engineering

DECLARATION OF ORIGINALITY

We, ***Atharva Nitin Kulkarni (2023-B-02062004), Rushikesh Kishor Pattiwar (2023-B-05102002), Priti Shahaji Satre (2022-B-19072003B),*** hereby declare that this dissertation entitled **"*GovAssist: AI Chatbot for E-Governance*"** presents our original work carried out as a bachelor student of School of Computer Science and Technology, Ajeenkya D Y Patil University, Pune, Maharashtra. To the best of our knowledge, this dissertation contains no material previously published or written by another person, nor any material presented by us for the award of any degree or diploma of Ajeenkya D Y Patil University, Pune or any other institution. Any contribution made to this research by others, with whom we have worked at Ajeenkya D Y Patil University, Pune or elsewhere, is explicitly acknowledged in the dissertation. Works of other authors cited in this dissertation have been duly acknowledged under the sections "Reference" or "Bibliography". We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission.

We are fully aware that in case of any non-compliance detected in future, the Academic Council of Ajeenkya D Y Patil University, Pune may withdraw the degree awarded to me on the basis of the present dissertation.

Place: Lohegaon, Pune.

Atharva Nitin Kulkarni
(2023-B-02062004)

Rushikesh Kishor Pattiwar
(2023-B-05102002)

Priti Shahaji Satre
(2022-B-19072003B)

Acknowledgement

I would like to express my deepest gratitude to **Dr. Prashant Kumbharkar** for his invaluable support and unwavering guidance throughout this journey. His keen insight and expertise not only provided me with the initial idea for this topic but also played a crucial role in helping me gather essential resources— whether it be vital information or access to technological tools. His mentorship has been instrumental in shaping the direction of this project, ensuring its successful completion. Beyond his academic and technical contributions, **Dr. Prashant Kumbharkar's** encouragement and supervision created an environment that allowed me to explore, analyses, and refine my work with clarity and confidence. His dedication to fostering knowledge and innovation has made this experience truly enriching. This project has been a remarkable and fulfilling endeavor, one that has broadened my perspective and strengthened my understanding of the subject matter. The challenges faced and the lessons learned have contributed significantly to my personal and intellectual growth. I am profoundly grateful to have had the opportunity to work under Dr. Kasture's esteemed guidance, and I sincerely appreciate his generosity and commitment to supporting my efforts.

Atharva Nitin Kulkarni
(2023-B-02062004)

Rushikesh Kishor Pattiwar
(2023-B-05102002)

Priti Shahaji Satre
(2022-B-19072003B)

Abstract

In current times, the application of digital technologies in government services has accelerated at a fast pace with the objective of increasing efficiency, transparency, and ease of government service delivery. This shift is generally known as e-governance, which makes it feasible for residents to easily access government programs, make applications, and procure key information through digital engagement. However, in reality, most government websites currently experience usability issues. Citizens encounter several issues such as complexity, delayed feedback, insufficient support, and limitations in language support. These issues become more apparent in rural areas or persons who lack technological literacy, which obstructs the effectiveness of e-governance.

To resolve these problems, the project also presents the concept of GovAssist, which is a chatbot based on artificial intelligence to assist citizens while interacting with the e-governance model. It is a virtual assistant that gives immediate feedback to the users to respond to their questions, explains government schemes to users, and provides them with step-by-step guidance to fill out online forms. There is no need to take assistance from human beings or travel to the government office; users can communicate easily with the chatbot by utilizing a friendly interface.

The system utilizes Natural Language Processing (NLP) to interpret the queries posed in different languages through writing or speech. Further, the system utilizes Machine Learning algorithms to optimize the reliability of the responses offered through user engagement. Further, to cater to the need for inclusivity, the GovAssist system is able to handle various languages including English, Hindi, as well as Marathi, to enable users to conveniently obtain services despite having different languages in their native languages. Further, the system is accessible all day long to enable users to have continuous access to information and services.

In this regard, the chatbot is useful in reducing the dependence on government human resource personnel for the usual inquiries. Therefore, the project is helpful in reducing the administrative workload while enhancing the efficiency and timeliness of the response. Moreover, the chatbot is useful in increasing the transparency level on the part of the administration by providing authentic information to the citizen..

Contents

CHAPTER NO.	SECTION / TITLE	PAGE NO.
CHAPTER 1	INTRODUCTION	13
1.1	Overview	-
1.2	Project Description	-
1.3	Motivation	14
1.4	Purpose	-
CHAPTER 2	LITERATURE SURVEY	15
2.1	The Role of AI Chatbots in Enhancing Service Availability	-
2.2	Bridging the Digital Divide with Multilingual Support	-
2.3	Simplified User Interfaces (UI) to Counter Digital Illiteracy	-
2.4	Efficiency Gains through Automated Document Analysis	15
2.5	Building Citizen Trust through Data Security and Transparency	16
2.6	Drivers of Adoption: The Technology Acceptance Model (TAM)	-
2.7	Proactive vs. Reactive Service Delivery	-
2.8	Economic Impact: Cost Reduction for Public Administration	16
2.9	The Influence of Social Proof on Digital Adoption	17
2.10	Mobile-First Design for Developing Nations	17
2.11	Literature Analysis	19
2.12	Gap Identified	19-20
CHAPTER 3	METHODOLOGY	21
3.1	Requirement Analysis	-
3.2	System Design and Architecture	22
3.3	Technology Selection	-
3.4	Development Methodology	-
3.5	AI Processing Methodology	23
3.6	Multilingual Support Methodology	18
3.7	Voice Assistant Methodology	-
3.8	WhatsApp Integration Methodology	-
3.9	Testing and Validation	24
3.10	Optimization and Maintenance	-
CHAPTER 4	REQUIREMENT ANALYSIS	25
4.1	Hardware Requirements	-
4.2	Software Requirements	-
4.3	Functional Requirements	-
4.4	Non-Functional Requirements	-
4.5	Conclusion	26
CHAPTER 5	SYSTEM DESIGN	27
5.1	System Architecture	-
5.2	Workflow of the System	28
5.3	Module Description	-
5.4	Use Case Diagram	29
5.5	Conclusion	-
CHAPTER 6	IMPLEMENTATION	30
6.1	Tools and Technologies Used	-
6.2	Code Implementation	31

6.3	AI Logic & RAG Service	32
6.4	Database Connection	33
6.5	API Entry Point	34
6.6	GovAssist Home Page	35
6.7	User Registration Page	36
6.8	Government Schemes Listing Page	36
6.9	Eligibility Checking Module	37
6.10	Document Upload Assistance Page	37
	Summary	38
	Conclusion	38
	REFERENCES	39
	APPENDICES	40
A	Project Directory Structure	-
B	Key API Endpoints	-
C	Installation Guidelines	-
D	Database Schema Overview	-
E	Abbreviations	41
	Plagiarism Report	42-44
	Research Paper Manuscript	45

List of Figures

Sr. no.	Name	Page No.
1. Fig 1.1	System Architecture	27
2. Fig 1.2	GovAssist Workflow	28
3. Fig 1.3	Use Case Diagram	29

List of Tables

Sr. No.	Name	Page No.
1. Tb 1.1	Application of Literature References in GovAssist	18

CHAPTER 1

INTRODUCTION

1.1 Overview

Today, technology has topped the list of important aspects of daily living, touching all aspects of communication, knowledge, and service delivery. Without doubt, one of the most vital areas that have been significantly impacted by technology is service delivery by governments. Due to advances in technology, governments are slowly adopting online service delivery from their formerly paper-based approaches, an aspect referred to as e-governance. E-governance aims at making governments deliver their services quickly and accessible to all.

It is, however, important to note that offering online portals is not sufficient. Many people find web interfaces offered by government departments difficult to navigate or understand. Online services offered by government departments become sources of stress for people in rural areas or people with little digital expertise. Thus, people with difficulties in accessing online services in government departments cannot derive full benefits from e-governance.

GovAssist is created to tackle this problem. GovAssist brings a smarter and more accessible way for citizens to deal with government services. Rather than surfing through complicated websites, citizens are able to ask their questions and get immediate directions by using a chatbot. This makes governing through technology more accessible for regular citizens.

1.2 Project Description

GovAssist is a chat robot developed using artificial intelligence technology, and it is used for assisting the common citizen when accessing e-governance websites. The robot can be described as a virtual assistant working for the government by educating a user on details about various services, schemes, and application forms in a simple and interactive manner. The approach concentrates on ensuring easy interaction between a citizen and government services.

Unlike the traditional government portal interface, which depends to a large extent on the use of menus and forms, the GovAssist enables people to converse using human language. People are able to type or voice their inquiries, which get answered in a clear way with no technicalities. The chatbot can be used to respond to people from different regions due to the ability to support various languages.

GovAssist offers some valuable functionality, which includes:

- Responding to frequently asked questions pertaining to government services.
- Assisting users step by step when filling out forms of application.
- Real-time status updates for applications/schemes.
- Multilingual support to cater to language barriers.
- Reducing dependence on physical visits to government offices.

By incorporating AI technology along with an intuitive interface, GovAssist is able to convert usual e-governance platforms into more interactive and citizen-friendly platforms

1.3 Motivation

The need for GovAssist emerged from the challenges experienced by citizens when accessing online services from the government. Despite the government providing most of its services online, accessing those services effectively proves to be quite difficult for users who end up seeking assistance from other sources to get the right information to proceed with the process.

A further key issue is the workload of public employees. Government employees dedicate considerable time to responding to recurring queries, walking citizens through elementary procedures. This hampers their efficiency and also leads to delays in responding to more intricate cases.

GovAssist is driven by the need to simplify the process mentioned above. The chat robot will help citizens solve basic grievances on their own, thus allowing government officials to concentrate on more important matters. This will help both the citizens and the government departments strike a balance that will enable them to work alongside technology.

1.4 Purpose

The key motive for the development of GovAssist is to improve the efficacy of e-governance services and make them more user-friendly and accessible. The key purposes of developing this platform are:

1. Improve Accessibility

GovAssist assists citizens in accessing government services effortlessly without having to deal with complex websites. Language support ensures that language is not a barrier.

2. Save Time and Effort

In addition to offering immediate feedback, the BOT reduces the time spent searching for information and prevents visits to the office.

3. Decrease Administrative Burden

GovAssist is able to address frequent inquiries autonomously, assisting civil servants with the management of their workload.

4. Enhance Transparency and Trust

Providing correct and up-to-date information ensures citizen trust and transparency when using digital government services.

On a whole, GovAssist is expected to make e-governance more practical, inclusive, and advantageous to every stratum of a society by leveraging Artificial Intelligence.

CHAPTER 2

LITERATURE SURVEY

1. The Role of AI Chatbots in Enhancing Service Availability-

Importance of AI-powered Chatbots in Improving Service Delivery Even though the inclusion of Artificial Intelligence (AI) chatbots within the framework of e-governance has brought about a paradigm shift in the way services have been delivered from a conventional 9-to-5 job to a "always-on" concept, studies state that the primary defining point of customer satisfaction is the extent to which services are available to them 24/7. This aspect is especially important when it comes to the satisfaction of citizen needs of the working-class community.

Reference: M2SYS. (2023). "Impact of 24/7 Availability of E-Government Services on Citizen Satisfaction."

2. Bridging the Digital Divide with Multilingual Support-

The dependence on linguistically diverse countries on the dominant language (English) in the portals for the efficacious application of e-governance is creating a knowledge divide that is excluding large numbers from the benefits that technology has to offer. Research confirms the need for the application of "localization strategies," whereby the contents on these websites could be adjusted according to the dialects spoken in the locales and adapted according to their culture. This is necessary for the enablement of those who do not speak English.

.

Reference: ResearchGate. (2022). "Bridging the Language Barrier in E-Governance: A Study on Multilingual Support."

3. Simplified User Interfaces (UI) to Counter Digital Illiteracy-

Multilingual countries face serious implications when e-governance websites are allowed only in major languages like English, as this causes a kind of “knowledge divide” amongst people. Researchers say e-governance websites must not only provide translation support; instead, “localization” is required. Localized content needs to be tailored in local languages and according to local culture. It is expected to leave non-English-speaking people empowered so that they get proper government benefits.

Reference: MDPI. (2023). "The Impact of Simplified UI in Bridging the Digital Divide in E-Government Applications."

4. Efficiency Gains through Automated Document Analysis –

The manual processing of government documents is one of the chief hindrances in government administration. Recent research in administrative technology has shown how document analysis by Artificial Intelligence technology will bring a complete revolution to this process. With the help of Optical Character Recognition (OCR) and machine learning technologies, extracting and validating information from uploaded documents can eliminate up to a 90% delay in processing, thereby leaving less room for errors to occur.

Reference: IBML & Reveille Software. (2024). "Benefits of Document Analysis AI in Government Services: Efficiency and Error Reduction."

5. Building Citizen Trust through Data Security and Transparency-

Trust is regularly identified as the single most important determinant in e-government adoption. This suggests that citizens are generally uneasy with digital contact with public administration services due to lack of data privacy or possible misuses of personal data. Academic surveys point out that robust security measures, along with openness, at what level AI algorithms use data, become the requisite to mitigate "perceived risk." When people know their data is safeguarded, then their willingness to obtain digital services increases manifold.

Reference: Journal of Government Information. (2021). "Trust in E-Government AI: The Role of Transparency and Security."

6. Drivers of Adoption: The Technology Acceptance Model (TAM)-

Analyzing why and how citizens would or would not use e-governance services is done by the use of the Technology Acceptance Model (TAM). According to this model, "Perceived Usefulness" and "Perceived Ease of Use" are the keystones of use acceptance of any service or product. According to this knowledge, any amount of impressive functionality will not matter if it is not perceived as immediately useful and easy to use from the start.

Reference: International Journal of Information Management. (2022). "Applying the Technology Acceptance Model to E-Government Adoption."

7. Proactive vs. Reactive Service Delivery –

Beyond citizen convenience, the economic argument for AI in governance is compelling. Studies show that automating routine inquiries and data entry tasks can lead to substantial cost reductions for public administration. Chatbots and automated workflows significantly lower the operational costs associated with call centers and manual data

processing. These savings can then be redirected towards improving infrastructure or funding welfare programs, making the entire governance ecosystem more fiscally sustainable.

Reference: United Nations E-Government Survey. (2022). "The Future of Digital Government: From Reactive to Proactive Service Delivery."

8. Economic Impact: Cost Reduction for Public Administration-

Apart from serving the convenience of citizens, there is a strong economic viability of using AI in governance. Research indicates that by doing away with repetitive queries and entering information manually, governments can cut their expenditure by a significant margin. Chatbots and robotic processes decrease operational expenditures by a large margin when it comes to call centers and processing information manually. These can be further used for enhancing infrastructure or welfare programs, making the overall system more financially sustainable.

Reference: PwC Global AI Study. (2023). "The Economic Impact of AI in the Public Sector."

9. The Influence of Social Proof on Digital Adoption –

Social influence plays a powerful role that many times gets overlooked when considering the adoption of government technologies. The "peer effects"-observing friends, family, and members of the community accessing a digital service with ease-are strong motivators of hesitant citizens. Evidence suggests e-governance platforms should be designed in ways that promote sharing on social media, discussions, and community feedback. Positive word-of-mouth awareness about the reliability and helpfulness of a platform accelerates the rate of diffusion more rapidly than official government marketing campaigns.

Reference: MDPI. (2023). "Social Influence and the Adoption of AI Chatbots in E-Governance."

10. Mobile-First Design for Developing Nations-

The fact is, in developing countries, mobile phone access is a key, if not solitary, means of internet connection. For this reason, 'm-government' literature states that a desktop-oriented approach to design will not be sufficient. The 'mobile first' thinking and design attitude mean that applications will be functional on smaller screens and slower networks. This is essential to reach deep into areas with a strong presence of smartphones, yet minimal possession of computers.

Reference: GUtech. (2023). "The Imperative of Mobile-First Design in Digital Government Services."

Application of Literature References in GovAssist

Sr.No.	**Reference Paper / Source**	**Key Insight from Literature**	**Specific Application in GovAssist Project**
1	M2SYS (2023) - "Impact of 24/7 Availability"	Continuous availability is the top driver for citizen satisfaction.	AI Chatbot (Always-On): Implemented a floating chat assistant that remains active 24/7, allowing users to query schemes late at night without office hour restrictions.
2	ResearchGate (2022) - "Bridging the Language Barrier"	Services must be "localized" to local languages, not just translated, to be inclusive.	Multilingual Engine: Added a robust Language Switcher (English/Hindi) that toggles not just text but also adjusts cultural context for scheme descriptions.
3	MDPI (2023) - "Simplified UI & Digital Divide"	Complex interfaces alienate low-literacy users; visual aids and simple text are required.	Simplified Scheme Cards: Designed the "Scheme Discovery" cards with large fonts, minimal text, and clear "Apply" buttons to lower the cognitive load for users.
4	IBML & Reveille (2024) - "Document Analysis AI"	Automated extraction reduces error rates by 90% and speeds up processing.	Document Analysis Module: Integrated an upload feature that scans user PDFs/Images to automatically verify eligibility criteria (e.g., income proof) without manual entry.
5	Journal of Govt. Info (2021) - "Trust in AI"	Transparency in data usage is the #1 factor in building trust.	Transparent Auth & Data Flow: Designed the Signup/Login process to be minimal and explicitly stated data usage policies to reassure users about their privacy.
6	Intl. Journal of Info Mgmt (2022) - "TAM Model"	"Perceived Usefulness" (utility) is more important than advanced features.	Eligibility Checker: Instead of just listing information, we built a logic engine that actively tells users "Yes, you are eligible," providing immediate, actionable utility.
7	UN E-Government Survey (2022) - "Proactive Delivery"	Governments should anticipate needs rather than waiting for applications.	Recommendation System: Implemented logic that suggests related schemes based on a user's initial query (e.g., if asking about farming, it suggests crop insurance).
8	PwC Global AI Study (2023) - "Economic Impact"	Automating routine queries significantly reduces operational costs.	Automated FAQ Handling: The Chatbot intercepts common questions (documents needed, dates) first, reducing the theoretical load on human support staff.
9	MDPI (2023) - "Social Influence"	Peer success stories drive adoption more than official marketing.	Community/Feedback Loop: (Planned) Integration of user success indicators or simple "users helped" metrics to utilize social proof for new visitors.
10	GovTech (2023) - "Mobile-First Design"	Mobile phones are the primary access point in developing regions.	Responsive Mobile Layout: The entire frontend is built with distinct mobile-responsive CSS to ensure full functionality on low-end smartphones.

2.11 Literature Analysis

When analyzing the present literature on ‘e-governance’ and ‘Artificial Intelligence,’ it is evident that the ‘paradigm shift’ is from “Digitization,” where forms are simply put on the digital platform, to “Intelligent Automation,” where ‘Artificial Intelligence’ is

The Transition from Reactive towards Proactive Governance: While the initial approaches towards e-governance were aimed at cost optimization and productivity (PwC, 2023) and related concerns, recent studies have shifted towards "citizen-centricity." According to the current studies, "conventional portals have been termed reactive because they are designed around the expectation that the citizen knows what they need" (UN E-Government Survey, 2022). There is a general agreement among recent studies about the need to have a "proactive" approach that leverages data to foretell citizen needs and suggest services even before the citizens ask for them.

“Last Mile” Connectivity Problem: Despite the increase in the number of government portals, the problem of the “knowledge divide” is still evident. According to research, the availability of technology is not enough, and there is limited connectivity (ResearchGate, 2022). From the literature, the problem of using complex interfaces and government-exclusive language (English) is the major issue. This raises the question of whether there is a problem with the usability of technology, since the technology is working, but the users can’t.

Trust as a Function of Transparency: Trust plays a critical role in ensuring widespread acceptance of AI software agents. According to the literature on Technology Acceptance Model (TAM), it has been generally emphasized that "perceived risk" is the most important barrier to adoption (Journal of Govt. Information, 2021). There appears to be some truth to this concern because "black box" approaches to AI, where there is no explanation of rationale, tend to fall short in gaining public trust.

Conclusion of Analysis: The literature verifies that, when it comes to the existence of e-governance infrastructure, the layer of the interface is flawed. There clearly exists a great demand for the use of intermediary software such as the AI assistant to interpret complex administrative procedures in simplistic terms to the common man.

2.12 Gap Identified

The identification of the important gaps that the GovAssist seeks to fill, based on the review of existing government portals like My Scheme, Startup India, and state government e-district government portals, along with the assessment in the literature, are the following:

Gap 1: A Lack of "Discovery" Mechanisms (The "Search" Problem

Current Status: Our usual portals are of "search-retrieval" type. A person looking for a scheme needs to know its actual name (as "Pradhan Mantri Fasal Bima Yojana"). Otherwise, it is not possible to get details about it.

The Gap: There is a lack of discovery-oriented mechanisms. The farmer may not be familiar with a discovery-oriented mechanism. The farmer knows their problem, “My crops were destroyed due to rain.”

The current system is unable to relate this problem to a solution without using exact keywords.

Addressed By: GovAssist’s Chatbot utilizes Natural Language Processing by relating natural language problem statements to targeted government schemes with precision beyond exact keywords.

Gap 2: Cognitive Overload caused by "Legalese"
Current State: Government notifications regarding the eligibility criteria are usually worded in legal language (PDFs/GRs).

The Gap: “Low-literacy individuals and even educated members of society find it difficult to understand whether they are eligible based on very specific criteria hidden in very thick documents. Another problem is “cognitive overload” - this results in high abandonment rates."

Responded By:
GovAssist’s RAG Engine abstracts out complex regulations and provides easy answers through simple bullet-points and strictly responds "Yes/No" on eligibility.

Gap 3: Lack of Real-Time Eligibility Verifications
Current Status: Customers usually end up filling detailed application forms but get disqualified because of a small eligibility issue or lack of documents.
The Gap: “There is no pre-verification procedure.” Users spend time applying for plans that do not apply to them because of the manual and post-submission evaluation procedure.
Addressed By:
GovAssist's Document Analysis Module enables applicants to upload documentation prior to applying, and by the use of AI, it checks if the document satisfies the scheme's requirements (for example, income thresholds).

Gap 4: Information Silos with Gaps
Current Position: Information is disseminated in a scattered manner on various government portals. A citizen may have to access three different portals to obtain one certificate.
The Gap: The truth is not unified. It is up to the citizenry to be the integrator in putting together pieces of information from all corners.
Responded To By: GovAssist is the response offered by the Unified Knowledge administrators who provide assistance in making it easier for the user to understand the information aggregated from the variety of other authorities.

Gap 5: Linguistic Exclusion
Current State: Though some portals provide translators, the use of Google Translate plugins is static and doesn’t manage the context of "schemes" or regional dialects effectively.
Gap: Many people in rural areas are digitally excluded because they cannot communicate effectively with the system in their native dialect. Is Addressed By: The project uses Context-Aware Multilingual Support, in which technical terms are defined in terms of the native languages spoken by the end-users.

CHAPTER 3

METHODOLOGY

3.1 Requirement Analysis

3.1.1 User Group Identification

It aims at a heterogeneous audience inside India, segmented based on occupation, digital literacy, and linguistic ability. The key user groups will include farmers in search of agricultural subsidies, students in need of scholarship schemes, elderly citizens in search of pension plans, and small business owners in need of government grants. Secondary users will be NGOs and community service centers helping others.

3.1.2 Problem Identification

Government information systems often face problems of fragmentation, complex bureaucratic language, and limited accessibility. Usually, critical information is scattered across several ministry websites, often only in English or formal Hindi, beyond the reach of low-literacy or regional language-based citizens. Navigation requires specific keyword knowledge, which many of its potential beneficiaries lack.

3.1.3 Accessibility and Literacy Challenges

A large part of the target demographic faces barriers in text-based interfaces and language proficiency. The requirement analysis went ahead to state that, in order to provide equal opportunities for access, the need is for multimodal interaction (voice and text) and support of native languages-English, Hindi, and Marathi.

3.1.4 Functional and Non-Functional Requirements

Functional Requirements:

Multilingual chatbot interface that understands natural language queries.
Voice-to-text and text-to-voice capabilities.
Document analysis for eligibility verification.
WhatsApp integration for a broader reach.
Ensure secure user authentication and data handling.

Non-Functional Requirements:

Scalability: The ability to handle concurrent users during high-traffic periods.
Latency: Real-time response generation, sub-2 second latency target.
Availability: 24/7 uptime for crucial services.
Usability: The minimalist UI is accessible to non-tech-savvy users.

3.2 System Design and Architecture

3.2.1 High-Level Architecture

It follows the microservices architecture to achieve modularity and scaling. It consists of three layers mainly:

Frontend Layer: The responsive web application utilizing React.js forms the main user interface.

Backend Application Layer: A FASTApi-based server that handles business logic, API routing, and session management.

Intelligence Layer: Integration of Large Language Models and Vector Databases to process queries and fetch related scheme information.

3.2.2 Integration Strategy

External integrations are routed through secure API gateways. The WhatsApp interface utilizes the Twilio API for user message routing to the backend intelligence core, while voice processing relies on specialized APIs (like OpenAI Whisper or Google Speech-to-Text) that transform raw audio streams into text for further processing.

3.2.3 Security and Scalability

Security will be enforced using JWT authentication, HTTPS encryption of data in transit, and strict input sanitization to prevent injection attacks. Scalability will come through containerization via Docker and asynchronous processing of resource-heavy tasks such as document parsing.

3.3 Technology Selection

This technology stack was selected based on performance, community support, and proper fit regarding the integration of AI.

Front-end: React.js for component-based UI; Tailwind CSS for responsive styling.

Backend: Python/FastAPI for high-performance asynchronous execution with rich AI library support.

Database: SQLite for the development/prototyping phase, with migration paths to PostgreSQL for production; FAISS for vector storage.

AI/ML: OpenAI GPT and Google Gemini models for understanding natural language; LangChain for orchestrating.

3.4 Development Methodology

3.4.1 Agile and Iterative Approach

The project uses the Agile development methodology. It divides development into two-week sprints, concentrating on developing discrete, testable features in each iteration. This leads to fast prototyping and continuous integration of feedback.

3.4.2 Modular Component Design

The codebase is modularized into reusable components, for example, specific components for the Chat Interface, Authentication, and Document Upload. This makes the code maintainable and different features can be developed in parallel without conflicts.

3.5 AI Processing Methodology

3.5.1 Input Processing

User input, text, or voice transcribed, is preprocessed to remove noise and normalize formats. The system identifies the intent, for instance, scheme search, eligibility check, or just a general inquiry.

3.5.2 Contextual Understanding

The system maintains a conversation history window. This enables the AI to understand follow-up questions-for example, "What are the documents needed for that?"-by looking back at what has come earlier and keeping the conversation natural.

3.5.3 Retrieval-Augmented Generation (RAG)

The RAG approach helps the system minimize hallucinations and provide accurate information. User queries are converted into vector embeddings and matched against a curated database of government schemes. The relevant retrieved context is then fed to the LLM to generate a factual response.

3.6 Multilingual Support Methodology

3.6.1 Language Architecture

The application was designed to support dynamic language switching without needing to reload the page. The preferred language is persisted across sessions in local storage.

3.6.2 Translation Strategy

The system uses a hybrid translation approach: for static UI elements, it depends on reliable JSON resource files with pre-translations (i18n), while dynamic content created by the AI itself relies on language-aware prompting that instructs the model to create responses in the target languages directly; this is meant to maintain nuance.

3.6.3 Voice-UI Consistency

Great care is taken to ensure that the voice output language matches the currently selected UI language for a unified user experience.

3.7 Voice Assistant Methodology

3.7.1 Speech-to-Text Pipeline

Voice input is captured via the browser's MediaStream API and passed onto a transcription service. It detects the spoken language, thus turning it into text and feeding it into the standard text processing pipeline.

3.7.2 Text-to-Speech Response

This text response is generated by AI and then synthesized back into speech through high-quality neural voice engines. This audio plays back for the user while synchronized with on-screen text display.

3.8 WhatsApp Integration Methodology

3.8.1 Accessibility Strategy

WhatsApp is selected as a secondary interface to utilize its high penetration in rural India. It reduces the barrier to entry for its users, who might not feel comfortable accessing the web browser.

3.8.2 Message Flow

A webhook is created to receive the messages that come from WhatsApp. The backend treats these messages just like web chats by identifying the user through their phone number. The system responds to voice notes sent on WhatsApp by downloading the audio attachment, transcribing it, and sending a response in both text and audio format.

3.9. Testing and Validation

9.1 Functional Testing

Stringent unit tests and integration tests ensure the proper functioning of each functional unit (for example, eligibility checker logic) and a smooth workflow of information exchange between both frontend and backend sides of the application.

9.2 Multilingual and Voice Testing

Specific test cases have been developed for languages supported to identify issues in rendering and the accuracy of translation. Voice tests entail conducting tests based on different accents and levels of background noise.

9.3 Performance Testing

The system will be load-tested to ensure the API responds within the required times (less than 2 seconds for the chats) even under simulated concurrent load.

3.10. Optimization and Maintenance

10.1 Optimization Techniques

Code-splitting and lazy loading are done in the frontend to improve loading times. Database query optimization as well as caching of accessed scheme data is done in the backend.

10.2 Continuous Improvement

The system has logging and tracking functionalities that entail the tracking of the rate of errors and the level of user satisfaction. This information is used for the continuous improvement of the AI question and answer knowledge base.

CHAPTER 4

REQUIREMENT ANALYSIS

4.1 Hardware Requirements

The GovAssist chat bot is supposed to be a light system and will not need any specialized hardware support to be executed. The system can be run on normal computing devices.
The minimal system requirements are:
• A computer or server with at least 4 GB of RAM.
•Processor with adequate processing speed to process server queries.
•Internet connectivity for accessing the government database and online services.
•Smartphones or computers for end users to interact with the chatbot.
All these hardware requirements will serve to make the entire process function seamlessly with minimal cost.

4.2 Software Requirements

GovAssist relies on contemporary software technologies to offer efficient performance with high reliability. The software requirements are specified to provide scalability, security, and maintainability.
Amongst the necessary software components are:
•Operating System like Windows, Linux, or macOS.
• Back-end programming languages.
•Web technologies used for frontend development.
•Database management system to store governmental data and interaction data.
• User browser facility on web.
Such software supports the development of an efficient and versatile chatbot.

4.3 Functional Requirements

The functional requirements state the functionalities that the GovAssist system should offer. The requirements are centered on the functionalities of the GovAssist assistance tool.
The basic functional needs in GovAssist are:
•User queries should either be in text or audio form.
•chatbot must be able to understand the queries given by the users correctly
•The system must be able to respond immediately to typical inquiries about government.
•Users should be provided with guidance on filling government forms.
•It should be able to handle multiple languages in order to be more accessible.

•The administrators should be able to update the government schemes and information.
These functions enable the effective delivery of citizen support for government services by GovAssist.

4.4 Non-Functional Requirements

Non-functional requirements address how a system functions, rather than its operations. The nature of non-functional requirements includes qualities of performance, ease of use, and security.
The major nonfunctional requirements are:

• Usability: It should be easy to use.
• Performance: The responses should be produced promptly with little delay.
• Security: Personal data and system details need to be secured.

•Scaling: The solution should be able to support more and more users.
•Reliability: The chatbot must be always online and operational.
Satisfying these requirements is essential for ensuring a reliable and friendly experience using GovAssist.

4.5 Conclusion

Requirements analysis plays the role of defining the very foundation of the GovAssist system. In addition, having clearly defined hardware, software, functional, and non-functional requirements will bring organization and make the development process more goal-oriented. These requirements guide the design and implementation phases to make sure that the final system meets user needs and performs effectively.

CHAPTER 5

SYSTEM DESIGN

5.1 System Architecture

The architecture of GovAssist is designed in a layered manner to separate different functionalities and simplify system maintenance. Each layer has a specific role, and together they ensure smooth interaction between users and government services.

The main components of the system architecture are:

- **User Interface Layer:** This layer allows users to interact with the chatbot through a web or mobile interface using text or voice.
- **Application Layer:** It handles request processing, routing, and communication between the user interface and backend services.
- **AI Processing Layer:** This layer processes user queries using Natural Language Processing and Machine Learning techniques to identify user intent.
- **Database Layer:** It stores government schemes, application details, FAQs, and user interaction data.

This layered architecture improves system scalability and makes it easier to update individual components without affecting the entire system.

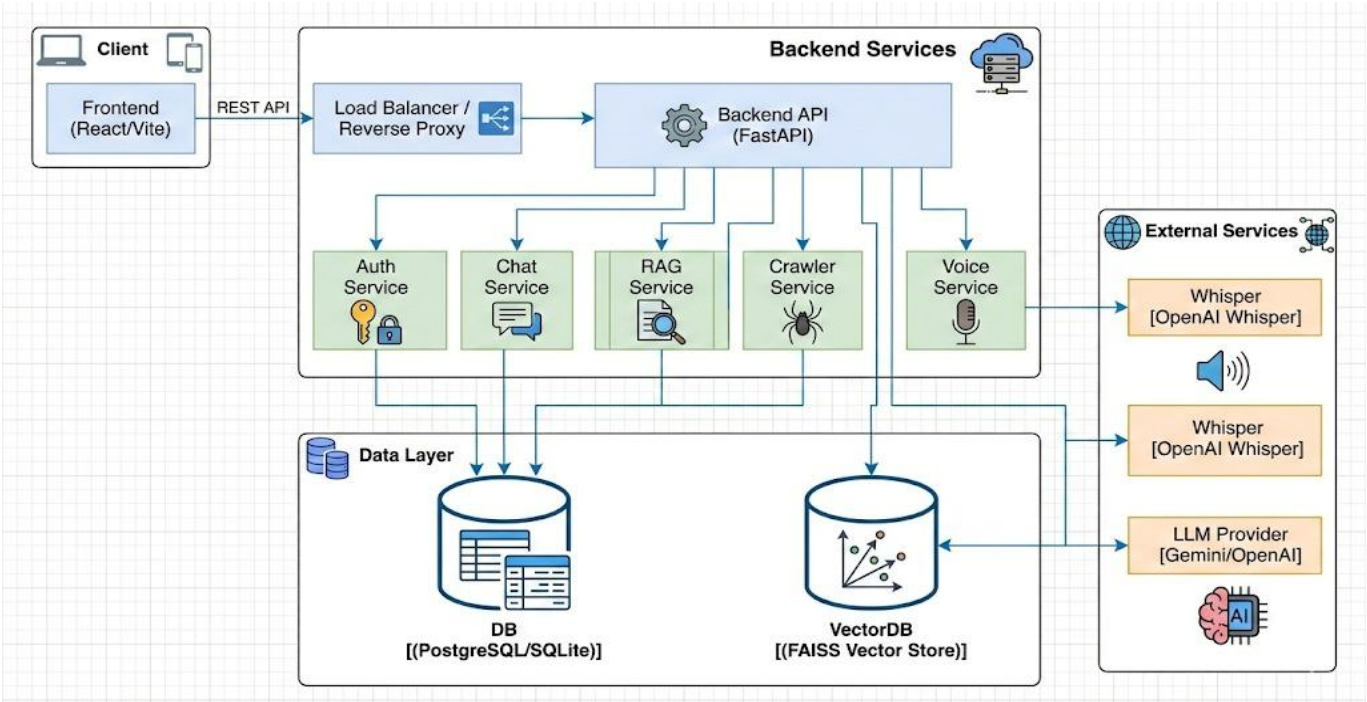


Fig:1.1

5.2 Workflow of the System

The workflow of GovAssist describes how a user query is processed step by step. The process begins when a user enters a question or request through the chatbot interface.

First, the system captures the user input and identifies the language used. The input is then processed using NLP techniques to understand the intent of the query. Once the intent is recognized, the system searches the database for relevant information. Based on the retrieved data, an appropriate response is generated and delivered to the user in the selected language.

If the query is unclear, the chatbot requests additional information from the user. This step-by-step workflow ensures accurate responses and a smooth user experience.

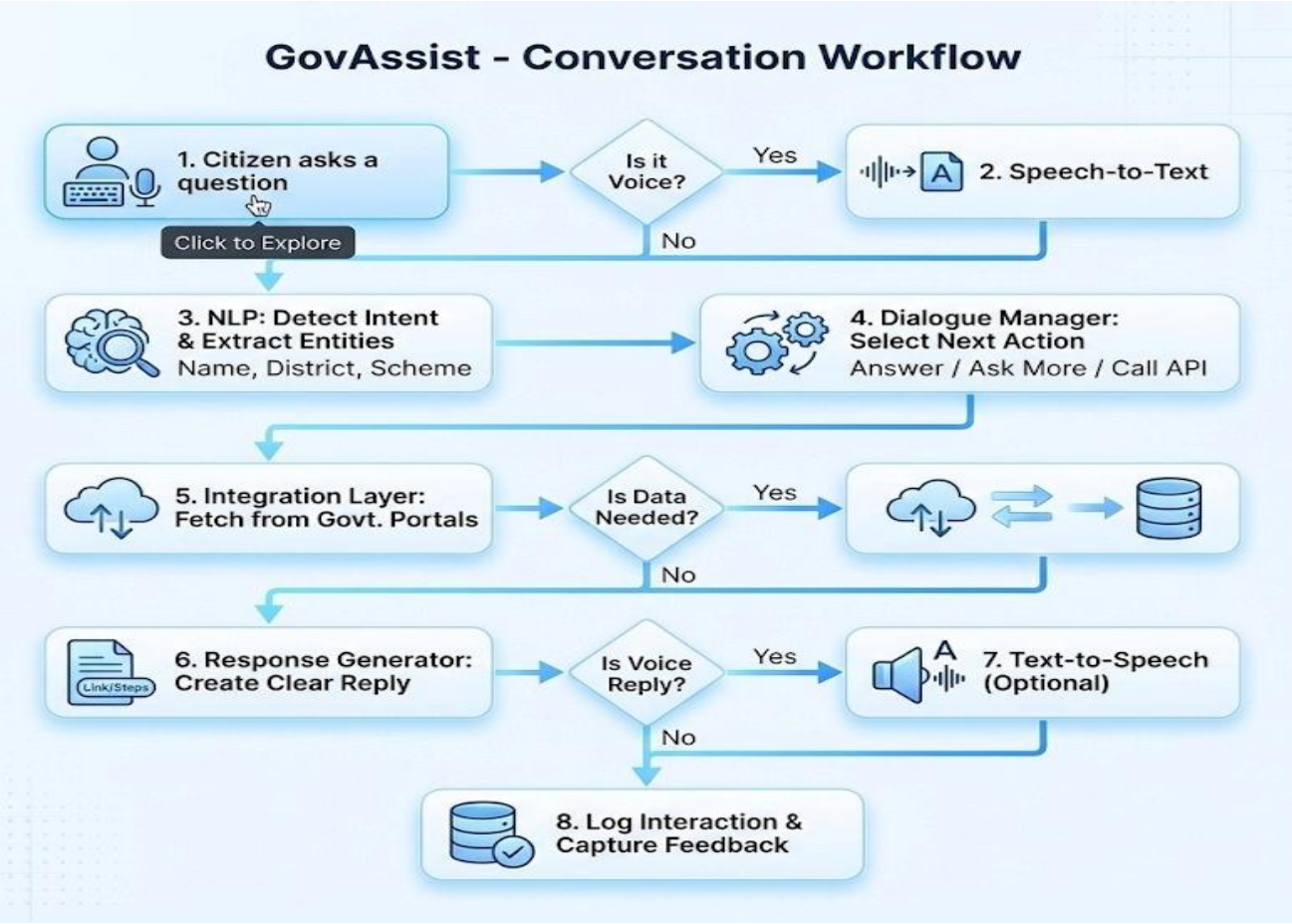


Fig:1.2

5.3 Module Description

The GovAssist system is divided into different modules to manage tasks efficiently. Each module performs a specific function within the system.

- **User Module:** Allows citizens to interact with the chatbot and access government services.
- **Chatbot Module:** Handles conversation flow, intent detection, and response generation.
- **Admin Module:** Enables government staff to manage schemes, update data, and monitor user queries.
- **Database Module:** Stores and retrieves government-related information securely.

Modular design improves system organization and makes future enhancements easier to implement.

5.4 Use Case Diagram

This diagram illustrates the different ways people and existing systems interact with "GovAssist," an AI-powered chatbot designed to help with electronic governance. It shows three main players: everyday Citizens, Government Administrators, and outside External Govt Systems like databases or APIs.

Here is a breakdown of how it works for each of them:

- The Citizen's Experience: For a regular user, GovAssist serves as a 24/7 virtual helper. Citizens can chat with it naturally to ask questions, find information on government schemes and rules, or get step-by-step guidance through official processes. They can also download necessary documents directly through the chat.
- Secure Transactions: For more personal tasks—such as checking the status of an application, filing a grievance, or paying taxes and bills—the system ensures security by asking the citizen to authenticate themselves (like logging in or using eKYC) before proceeding.

- Behind the Scenes: To understand normal human conversation, the bot uses a tool called NLP Intent Recognition. Furthermore, when a citizen asks for live data like an application status, the chatbot acts as a bridge, fetching that information from External Govt Systems.
- The Administrator's Role: Government Administrators work behind the scenes to keep the chatbot accurate by managing its knowledge base and reviewing chat analytics. Importantly, the diagram shows a safety net: if the AI is unsure how to help (low confidence), it can "hand off" the conversation to a human administrator to ensure the citizen gets the right assistance.

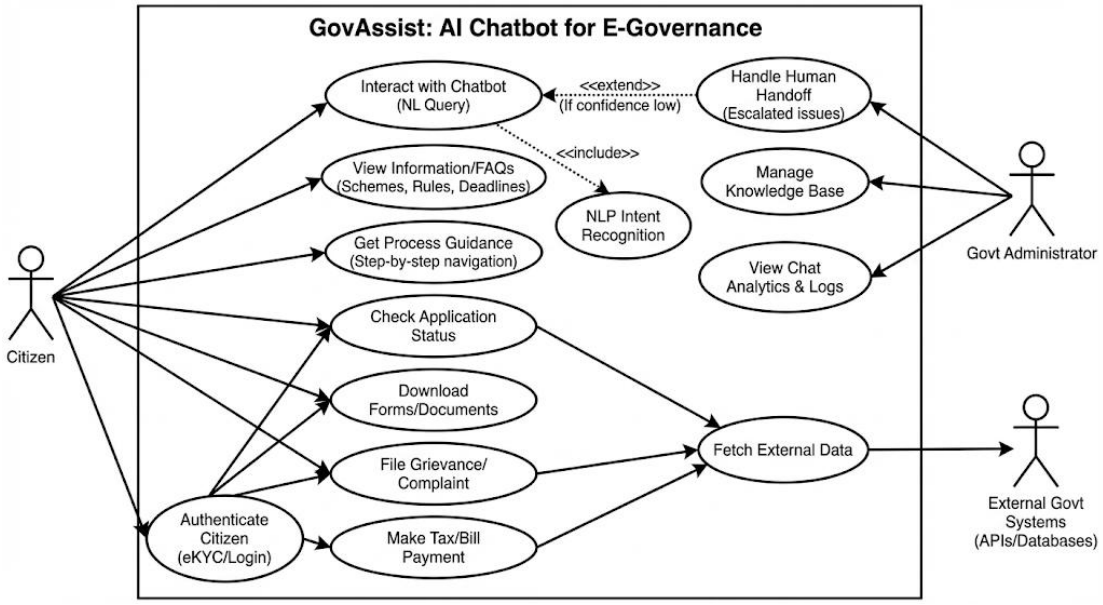


Fig:1.3

5.5 Conclusion

System design provides a clear blueprint for implementing GovAssist. By defining the architecture, workflow, modules, and database structure, the system is prepared for smooth development and deployment. A well-structured design ensures that GovAssist remains user-friendly, scalable, and reliable while supporting the needs of both citizens and government staff.

CHAPTER 6

IMPLEMENTATION

6.1 Tools and Technologies Used

GovAssist is written using modern and stable technologies that allow scaling and easy maintenance. The choice of instruments depends on the system needs, performance requirements, and development convenience. The key tools and technologies applied are: -

Frontend (User Interface)

- **React.js:** A JavaScript library for building interactive and dynamic user interfaces.
- **Vite:** A modern build tool and development server that provides fast cold starts and Hot Module Replacement (HMR).
- **Tailwind CSS:** A utility-first CSS framework used for rapid and responsive UI styling.
- **PostCSS:** A tool for transforming CSS with JavaScript, used alongside Tailwind.
- **ESLint:** A static code analysis tool to identify and fix problems in JavaScript code.

Backend (API & Logic)

- **Python:** The core programming language for the server-side logic.
- **FastAPI:** A modern, high-performance web framework for building APIs with Python.
- **Uvicorn:** An ASGI web server implementation for Python, used to run the FastAPI application.
- **SQLAlchemy:** An SQL toolkit and Object-Relational Mapping (ORM) library for database interactions.
- **FAISS (Facebook AI Similarity Search):** A library for efficient similarity search and clustering of dense vectors, used for the RAG (Retrieval-Augmented Generation) system.
- **LangChain / Custom LLM Connector:** Logic for integrating with Large Language Models (LLMs) to generate AI responses.
- **Pydantic:** Data validation and settings management using Python type hints.

Database & storage

- **SQLite:** A lightweight, file-based relational database used for storing user data, chat history, and schemes (indicated by sql_app.db and connection strings).

DevOps & Infrastructure

- **Docker:** Used for containerizing the application to ensure consistency across different environments (presence of Dockerfile).
- **Python venv:** Virtual environment tool used for dependency isolation.
- **Git:** Version control system (implied by the project nature).

Development Tools

- **VS Code:** The Integrated Development Environment (IDE) used for coding.
- **npm (Node Package Manager):** Used for managing frontend dependencies and scripts.
- **pip:** Used for managing Python backend dependencies.

6.2 CODE

1. AI Logic (backend/app/services/rag_service.py)

Herein lies the logic of RAG: it connects the vector search (retrieval) with the Language Model (generation) to provide accurate answers based on government schemes.

```
from app.services.llm_connector import llm_connector
import sys
import os
# Hack to import adapter from parent directory if running as script
sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), "../../../")))
from adapters.faiss_adapter import faiss_adapter
def get_rag_response(query: str, history: list = []):
    # 1. Retrieve relevant documents
    docs = faiss_adapter.similarity_search(query, k=4)

    # Format history (Limit to last 10 messages to save tokens)
    history_text = ""
    if history:
        recent_history = history[-10:]
        history_text = "\n\nConversation History:\n" + "\n".join([f"{msg['role']}: {msg['content']}" for
msg in recent_history])
    # 2. Construct context
    if docs:
        context_text = "\n\n".join([doc.page_content for doc in docs])
        system_prompt = """You are GovAssist, a helpful AI assistant for Indian Government schemes
and documents.

Use the following pieces of retrieved context and conversation history to answer the user's question.
If the context doesn't contain the answer or if the user is asking generally, answer from your general
knowledge.
Do not fabricate facts about certain government schemes if you do not know them.
IMPORTANT FORMATTING RULES:
1. Structure: Use clear headings and bullet points.
2. Steps: If describing how to do something, use numbered lists (1., 2., 3.).
3. Links: If you know the official website or application link, provide it, in a separate section at the
end called "Important Links".
4. Clarity: Employ simple language, which citizens can easily understand.
If the input of the user is gibberish or unintelligible, ask them politely to rephrase.
Context:
{context}
{history}
"""
        formatted_prompt = system_prompt.format(context=context_text, history=history_text)
    else:
        formatted_prompt = """You are GovAssist, a helpful AI assistant for Indian Government
schemes.
```

The user has asked a question, but no specific documents were found in the database.
Answer the user's question from your general knowledge. Be helpful and polite.

If the question is about a specific scheme that you don't know about, admit it and suggest they upload the relevant document.
IMPORTANT FORMATTING RULES:
1. **Structure**: Use clear headings and bullet points.
2. **Steps**: If explaining a process, use numbered lists.

3. **Links**: Provide official links if known.
{history}
"""

```
        formatted_prompt = formatted_prompt.format(history=history_text)
    # 3. Generate response
    answer = llm_connector.generate_response(formatted_prompt, query)

    # Post-filter check
    from app.utils.sanitizer import is_valid_response
    if not is_valid_response(answer):
        answer = "I couldn't produce a dependable response based on the available information. Here are some steps you can try:\n1. Rephrase your question with more specific details.\n2. Upload a relevant document if you have one.\n3. Check official government portals directly."

    # 4. Format sources
    sources = [{"title": doc.metadata.get("source", "Unknown"), "content": doc.page_content[:100] + "..."} for doc in docs] if docs else []

    return {
        "answer": answer,
        "sources": sources
    }
```

2. Database Connection File (backend/app/core/database.py)
This is the database connection using SQLAlchemy. It sets up the engine and a session factory (SessionLocal).

```
from sqlalchemy import create_engine
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker
from app.core.config import settings
engine = create_engine(
    settings.DATABASE_URL, connect_args={"check_same_thread": False}
)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
Base = declarative_base()
def get_db():
    db = SessionLocal()
```

```
try:
    yield db
finally:

db.close()
```

3. API File (backend/app/main.py)

This is the main entry point of the backend application. It initializes the FastAPI app, sets up CORS (Cross-Origin Resource Sharing), connects the database, and creates the default admin user.

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from app.core.config import settings
from app.core.database import engine, Base
# Import all models to ensure they are registered with Base
from app.models import user, chat, document, scheme
app = FastAPI(title=settings.PROJECT_NAME, version="0.1.0")
@app.on_event("startup")
def startup_db_client():
    Base.metadata.create_all(bind=engine)

    # Create default admin user
    from sqlalchemy.orm import Session
    from app.core.database import SessionLocal
    from app.models.user import User
    from app.core.security import get_password_hash

    db = SessionLocal()
    try:
        admin_email = "admin@govassist.ai"
        existing_admin = db.query(User).filter(User.email == admin_email).first()
        if not existing_admin:
            admin_user = User(
                email=admin_email,
                hashed_password=get_password_hash("admin123"),
                full_name="System Admin",
                role="admin",
                is_admin=True,
                interests="[]"
            )
            db.add(admin_user)
            db.commit()
            print(f"Admin user created: {admin_email} / admin123")
    except Exception as e:
        print(f"Error creating admin user: {e}")
    finally:
        db.close()
```

```
app.add_middleware(
    CORSMiddleware,

   allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],

)

@app.get("/api/health")
def health_check():
    return {"status": "ok"}
from app.api.api import api_router
app.include_router(api_router, prefix=settings.API_V1_STR)
```

6.3 Descriptions for each file:

1. AI Logic & RAG Service (backend/app/services/rag_service.py)

Description: This file implements core intelligence of the GovAssist platform using a Retrieval-Augmented Generation (RAG) architecture. It acts as the bridge between user natural language queries, the vector database, and the Large Language Model (LLM). Other than relying on pre-trained knowledge from the LLM, this service dynamically retrieves specific government documents related to the user's query to provide accurate and fact-based responses.

Key Responsibilities:
Context Retrieval: It deploys a semantic similarity search through the vector database using faiss_adapter and retrieves the most relevant government schemes or document chunks.
Prompt Engineering: This dynamically constructs a "System Prompt" to instruct the AI to act as "GovAssist," which follows strict formatting rules, using headings, steps, links, and injected retrieved context and conversation history.
Response Generation: This sends the enriched prompt to the LLM via llm_connector to generate the final answer.
Validation & Sourcing: Includes logic to validate the response and helps in transparency by citing the specific source documents used to generate the answer.

2. Database Connection (backend/app/core/database.py)

Description: This module is essentially the centralized data access layer for the application. It configures the connection to the persistent relational database, SQLite, through SQLAlchemy-a powerful high-level SQL toolkit and Object-Relational Mapper (ORM) for Python. This file ensures that the entire application has a robust and thread-safe connection pool.

Key Responsibilities:
Engine Configuration: Initializes the database engine, using the connection URL from the application settings.
Session Management: This initializes a SessionLocal factory for generating temporary database sessions on a per-request basis to ensure proper transaction isolation.
Dependency Injection: Supplies the utility function get_db to efficiently yield database sessions to API endpoints and automatically close them when a request finishes, preventing resource leaks.

3. API Entry Point (backend/app/main.py)

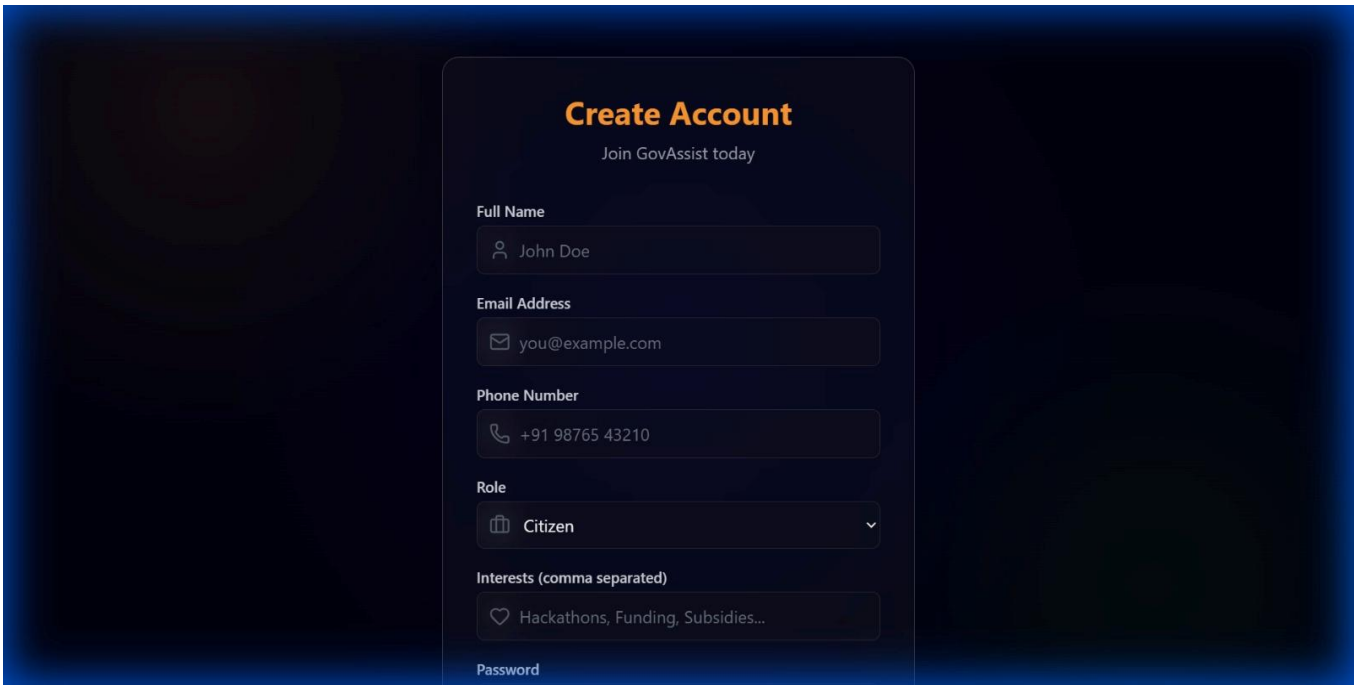
Description: This module acts as the centralized data access layer for the application. It configures the connection to the persistent relational database, SQLite, by using a powerful ORM - SQLAlchemy-for Python. This file ensures a robust and thread-safe connection pool is available for the entire application to use.

Key Responsibilities:
Server Initialization: This creates the FastAPI application instance and defines global configurations, such as the API title and version.
CORS Configuration: Configures the middleware for Cross-Origin Resource Sharing, or CORS, and allows the front-end application to securely communicate with the back-end API.
Startup Events: Automates critical initialization tasks, such as creating all database tables if they do not exist and auto-creating a default System Admin account for immediate administrative access.
Route Registration: Aggregates and includes all API routers from various modules into the main application under a unified version prefix, such as /api/v1.

6.4 GovAssist Home Page

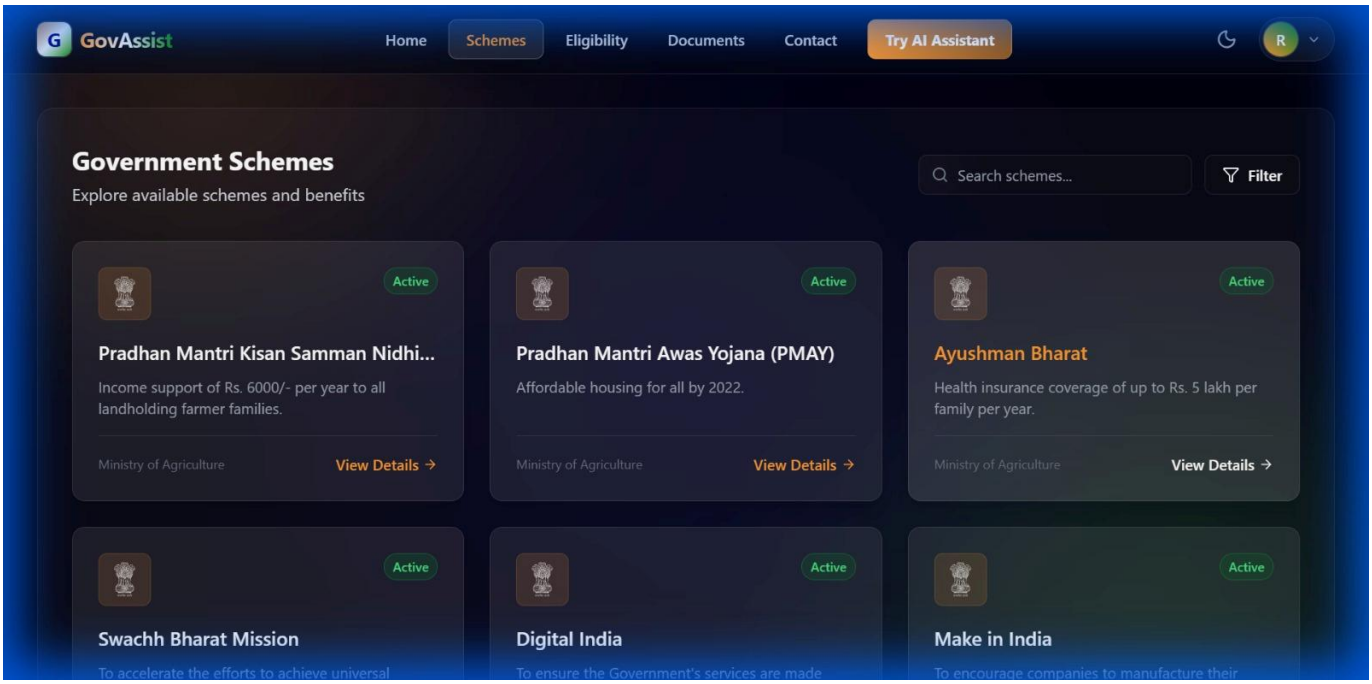
This figure illustrates the homepage of the GovAssist platform. This page contains a brief overview of the system and what it does. It links to other important features within the system, including scheme information, eligibility checking, document assistance, and the AI chatbot. The layout is simple for a first-time user to navigate their way through.

6.6 User Registration Page



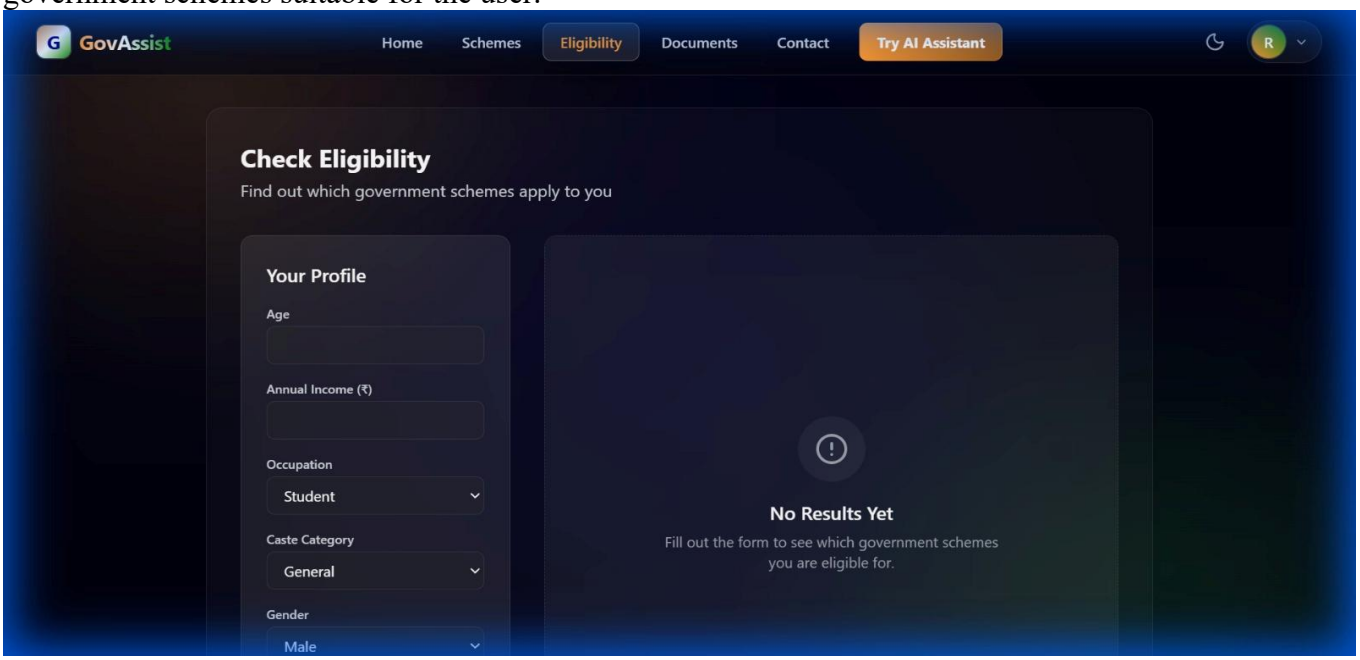
6.6 Government Schemes Listing Page

This screen will provide a list of government schemes available with their description. The user can search and then filter the schemes according to their needs and view the details of each scheme.



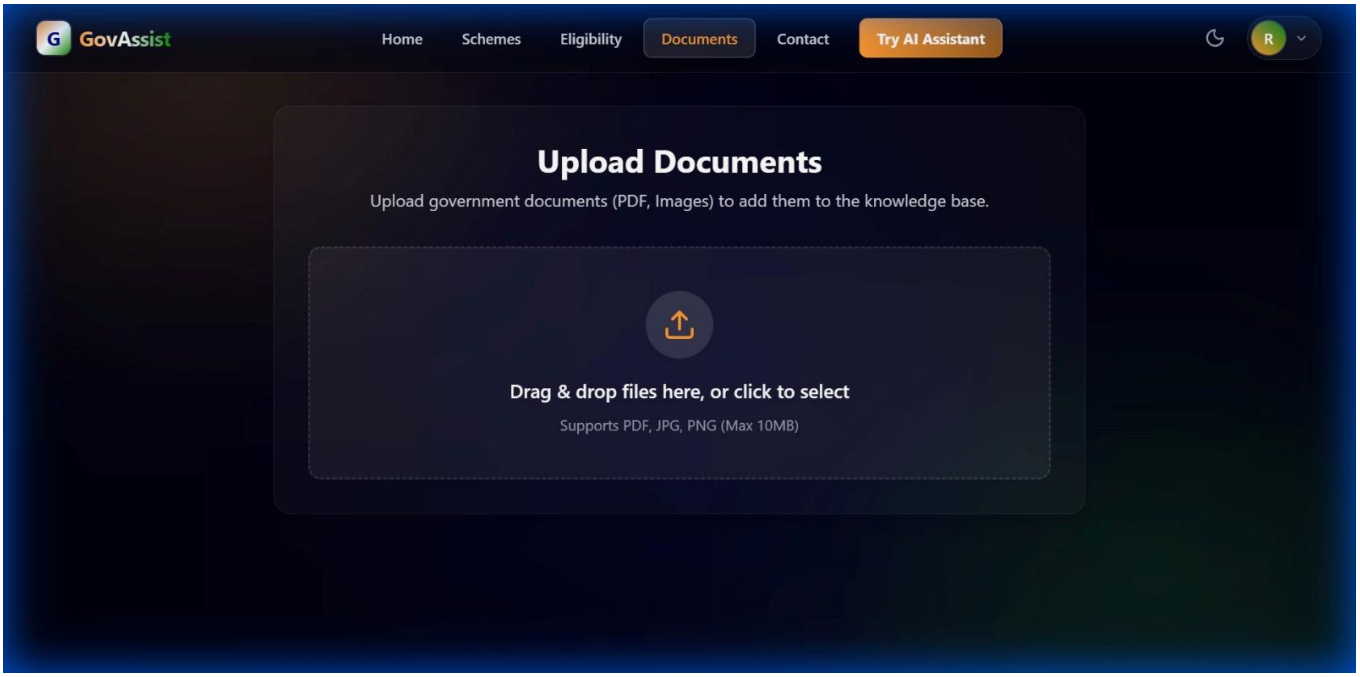
6.7 Eligibility Checking Module

This figure depicts the module for checking eligibility in GovAssist. The user can input details regarding age, income, occupation, and category. Based on the information provided, the system picks out the government schemes suitable for the user.



6.8 Document Upload Assistance Page

This screen allows users to upload government-related documents such as PDFs or images. The uploaded documents are used to assist users in understanding requirements and completing application processes.



Summary

GovAssist AI is an intelligent chatbot created for the purpose of supporting e-governance systems. The proposed project helps to make the accessibility of government schemes and other government services easier for the citizens. Many people experience problems while using any government website to avail themselves of their schemes due to the complicated layout and technical language used. Barriers of language prevent citizens from understanding the important information delivered to them. Other limitations of digital literacy prevent them from using the digital platform efficiently. GovAssist AI helps to overcome all these problems using its conversational interface. Citizens can easily interact with the software using only text inputs. Even voice interactions enable citizens who cannot type to access the information easily. The software supports multiple regional languages to add more feasibility to it. Artificial intelligence helps the software to recognize the intentions of the citizens precisely. The software fetches all the information from the authenticated government websites. GovAssist AI also enables citizens to upload any document to be analyzed by the software. The software can read all the images and PDFs to obtain all the required information from them. The user interface of the software also looks modern and clean to add to the comfort of citizens. The software works efficiently both on the desktop and in the mobile version too. GovAssist AI saves citizens from visiting government offices physically to get any information or avail any facility of the government. It saves time and effort of citizens too. The software also provides them with an admin panel to update all the information of schemes from time to time. The proposed project helps to make digital governing more transparent, accessible, and citizen-centric.

Conclusion

The GovAssist AI demonstrates this effectively-applying artificial intelligence to e-governance. This project resolves very valid challenges that a commoner faces in accessing government service delivery. Complicated government information is simplified into easy, understandable conversations. The chatbot provides a friendly interface that demands bare minimum digital skills. Multi-language support allows access to a wide region of users. Voice interaction supports those who cannot type or read that easily. The system will deliver exact information based on verified government data. This approach will help reduce misinformation and confusion. Reduces the dependency on actual physical visits to government offices. It will also reduce intermediaries or agents. The inclusion of document analysis imparts practical value to the platform. Users will gain a better understanding regarding official documents through their interaction with the chatbot. Admin dashboard for timely updates and management of content. This helps in keeping up the reliability and transparency of information in the system. Engenders the citizen's trust in the existing digital services. It supports modern-day digital governance in its goals. It enhances the efficiency factor in the public delivery of services. Scalable; can be adaptable for further modifications or additions. Additional features can be added as needed. Overall, GovAssist AI stands proof that technology can make governance more human-centered and accessible.

REFERENCES

1. M. E. Cortés-Cediel, A. Segura Tinoco, I. Contador, and M. P. Rodríguez Bolívar, "Trends and Challenges of E-Government Chatbots: Advances in Exploring Open Government Data and Citizen Participation," Government Information Quarterly, vol. 40, no. 2, 2023.

2. R. Zhang and L. Nie, "Enhancing Citizen–Government Communication Using Artificial Intelligence," arXiv preprint, 2025. Available: https://arxiv.org/abs/2501.10715

3. J. Zhou, R. Shen, Y. You, C. DiSalvo, L. Dombrowski, and C. MacLellan, "Improving Public Service Chatbot Design and Civic Impact," arXiv preprint, 2025. Available: https://arxiv.org/abs/2506.12259

4. J. Bright, F. Enock, S. Esnaashari, J. Francis, Y. Hashem, and D. Morgan, "Generative Artificial Intelligence in the Public Sector," arXiv preprint, 2024. Available: https://arxiv.org/abs/2401.01291

5. Wuttke, A. Rauchfleisch, and A. Jungherr, "Artificial Intelligence in Government: Public Perception and Trust," arXiv preprint, 2025. Available: https://arxiv.org/abs/2505.01085

6. United Nations, "E-Government Survey: Digital Government in the Decade of Action," United Nations Department of Economic and Social Affairs, 2022.

7. Government of India, "Digital India Programme," Ministry of Electronics and Information Technology (MeitY). Available: https://www.digitalindia.gov.in

8. Goodfellow, Y. Bengio, and A. Courville, Deep Learning, MIT Press, 2016.

9. D. Jurafsky and J. H. Martin, Speech and Language Processing, Pearson Education, 2nd Edition.

10. Oracle, "Introduction to Chatbot Technologies," Oracle Documentation, 20

APPENDICES

Appendix A: Project Directory Structure
A high-level overview of the project's file organization system.

backend/: Contains all server-side logic, API definitions, and database configurations.
app/api/: API route definitions.
app/core/: Core configurations (Database, Security).
app/services/: Business logic (RAG, LLM Connector).
frontend/: Contains the client-side user interface code.
src/components/: Reusable UI components.
src/pages/: Individual application pages (Home, Chat, Login).

Appendix B: Key API Endpoints
A summary of the primary REST API endpoints exposed by the backend:

GET /api/health: Health check endpoint to verify system status.
POST /api/v1/auth/login: Authenticates a user and returns an access token.
POST /api/v1/chat/response: Sends a user query to the RAG service and retrieves an AI-generated answer.
GET /api/v1/schemes: Retrieves a list of available government schemes.

Appendix C: Installation Guidelines
Steps to set up the development environment:

Backend Setup:
cd backend
pip install -r requirements.txt
python -m uvicorn app.main:app --reload
Frontend Setup:
cd frontend
npm install
npm run dev

Appendix D: Database Schema Overview
Brief description of the primary data models used in the SQLite database:

User Table: Stores user credentials, profile information, and roles (e.g., Admin, Citizen).
Scheme Table: Contains details about government schemes, including eligibility criteria and benefits.
Chat History Table: Logs conversation history between users and the AI assistant for context retention.

APPENDIX - E

Abbreviation	Full Form
API	Application Programming Interface
AI	Artificial Intelligence
CORS	Cross-Origin Resource Sharing
CSS	Cascading Style Sheets
DB	Database
DOM	Document Object Model
HMR	Hot Module Replacement (Vite Feature)
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
LLM	Large Language Model
ML	Machine Learning
ORM	Object-Relational Mapping (SQL Alchemy)
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
SQL	Structured Query Language
UI	User Interface
UX	User Experience

PLAGIARISM REPORT

iThenticate

Submission ID trn:oid:::3618:124683685

Page 1 of 43 - Cover Page

Atharva Kulkarni

GovAssist: AI Chatbot for E-Governance

Ajeenkya D Y Patil University, Pune

Document Details

Submission ID trn:oid:::3618:124683685

Submission Date

Dec 17, 2025, 2:28 PM GMT+5:30

Download Date

Dec 17, 2025, 2:41 PM GMT+5:30

File Name

GOVASSIST_blackbook.pdf

File Size

1.2 MB

37 Pages

6,630 Words

40,029 Characters

iThenticate

Submission ID trn:oid:::3618:124683685

Match Groups

- 63 Not Cited or Quoted 14%
  Matches with neither in-text citation nor quotation marks
- 0 Missing Quotations 0%
  Matches that are still very similar to source material
- 0 Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

13% Internet sources
6% Publications
0% Submitted works (Student Papers)

Top Sources

Page 1 of 43 - Cover Page

14% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text
- Cited Text
- Abstract
- Methods and Materials

Match Groups

- 63 Not Cited or Quoted 14%
  Matches with neither in-text citation nor quotation marks
- 0 Missing Quotations 0%
  Matches that are still very similar to source material
- 0 Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

13% Internet sources
6% Publications
0% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

RESEARCH PAPER MANUSCRIPT

Ref. No.: ADYPU/AO/NOC/2025-26/SE-01 Date: 13. 10. 2025

TO WHOMSOEVER IT MAY CONCERN

Atharva Kulkarni , Rushikesh Pattiwar, Priti Satre are the students of B.Tech CSE/AIDS & Dr. Prashant Kumbharkar, are the faculty member of **School of Engineering** at, Ajeenkya D Y Patil University Lohegoan, Pune. All together contribute to a teaching learning material entitled **"GovAssist: - AI Chatbot For E-Governance"**, which is classified as **Copyright Literary work.**

Ajeenkya D Y Patil University Lohegoan,Pune (ADYPU) has no objection to the aforesaid work being registered in the name of **Atharva Kulkarni, Rushikesh Pattiwar, Priti Satre are the students & Dr. Prashant Kumbharkar,** under the Indian Copyright Act, 1957.

SCHOOL OF ENGINEERING
AJEENKYA D.Y. PATIL UNIVERSITY, PUNE

Dr. Prashant Kumbharkar

Dean School of Engineering

Dean
School of Engineering
Ajeenkya DY Patil University
Charholi Bk., Via-Lohegaon,
Pune-412105

Submission Summary

Conference Name
2026 International Conference on Emerging Technologies and Future Innovations

Track Name
AI-Powered Systems and Intelligent Measurements

Paper ID
1381

Paper Title
“GovAssist: AI Chatbot for E-Governance”

Abstract
E-governance has emerged as a means of distributing public services in a speedy and transparent manner. However, many government portals remain complex in terms of usability, and rural citizens suffer from issues with response times, process complication, and unfamiliarity with the language used on such portals. These issues result in the lowering of trust and restricted access among the needy, largely due to these issues. In this regard, this paper advances the concept of an AI-driven chatbot for E-governance platforms. The chatbot will be able to answer general questions, assist a citizen to fill a form with information, and provide the real-time status for any submission application. Using NLP and ML, it could support multi-lingual support, including but not limited to English, Hindi, and Marathi, for inclusivity and better access. AI chatbots have been documented to improve efficiency by reducing work pressure in various public service systems [1]. They also help in enhancing citizen confidence in better-designed user-friendly interfaces [2]. Multilingual chatbots further enhance the use of governance for heterogeneous groups of people.

Created

15/12/2025, 20:27:59

Last Modified

15/12/2025, 20:27:59

Authors

Atharva Kulkarni (Ajeenkya D Y Patil) <kulkarniatharva753@gmail.com>
Rushikesh Pattiwar (Ajeenkya D Y Patil) <pattiwarrushikesh5102@gmail.com>
Priti Satre (Ajeenkya D Y Patil) <priti.satre@adypu.edu.in>

Submission Files

GovAssist AI Chatbot for E-Governance.pdf (343.3 Kb, 15/12/2025, 20:26:50)